Combining SIMD and Many/Multi-core Parallelism for Finite State Machines with Enumerative Speculation

Peng Jiang Gagan Agrawal

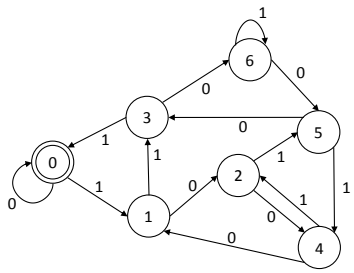
Department of Computer Science and Engineering
The Ohio State University
Columbus, OH 43210, USA
{jiang.952, agrawal.28}@osu.edu

Abstract

Finite State Machine (FSM) is the key kernel behind many popular applications, including regular expression matching, text tokenization, and Huffman decoding. Parallelizing FSMs is extremely difficult because of the strong dependencies and unpredictable memory accesses. Previous efforts have largely focused on multi-core parallelization, and used different approaches, including *speculative* and *enumerative* execution, both of which have been effective but also have limitations. With increasing width and improving flexibility in SIMD instruction sets, this paper focuses on combining SIMD and multi/many-core parallelism for FSMs. We have developed a novel strategy, called *enumerative speculation*. Instead of speculating on a single state as in speculative execution or enumerating all possible states as in enumerative execution, our strategy speculates transitions from several possible states, reducing the prediction overheads of speculation approach and the large amount of redundant work in the enumerative approach. A simple lookback approach produces a set of guessed states to achieve high speculation success rates in our enumerative speculation. We evaluate our method with four popular FSM applications: Huffman decoding, regular expression matching, HTML tokenization, and Div7. We obtain up to 2.5x speedup using SIMD on one core and up to 95x combining SIMD with 60 cores of an Intel Xeon Phi. On a single core, we outperform the best single-state speculative execution version by an average of 1.6x, and in combining SIMD and many-core parallelism, outperform enumerative execution by an average of 2x.

1. Introduction

The essential computations in many practical applications, including regular expression matching [1], dictionary-base decoding, XML validation, text tokenization, and network intrusion detection, can be expressed as state transitions on finite state machines (FSMs). Thus, the performance of FSMs is critical to the performance of these popular applications. Figure 1a shows an example FSM that checks if a binary number is divisible by 7. This FSM has 7 states labeled from 0 to 6, indicating the remainder when the number is divided by 7. From a given state i, we transition to state $(2i \; mod \; 7)$ after processing a 0, and to state $((2i+1) \; mod \; 7)$ after processing a 1. A straightforward implementation of the state transition is shown in Figure 1c, and the state transition table is shown in Figure 1b.



(a)

input \ state	0	1	2	3	4	5	6
Bit 0	0	2	4	6	1	3	5
Bit 1	1	3	5	0	2	4	6

(b)

```
state = 0;
for(i in input_text) {
  state = T[i][state];
}
```

(c)

Figure 1: An example of FSM and its state transition

An FSM is clearly an example of irregular application because of the data-dependent execution. They have even been referred to as *embarrassingly sequential* applications, as it is extremely hard to parallelize them [1]. More specifically, the iterations in the state transition loop are data-dependent because the next iteration must wait for the current iteration to produce the state. In addition, the memory accesses are non-continuous and input-dependent, i.e., the address of the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PPoPP '17, February 04-08, 2017, Austin, TX, USA

© 2017 ACM. ISBN 978-1-4503-4493-7/17/02…$15.00

DOI: http://dx.doi.org/10.1145/3018743.3018760

state read from the state transition table is determined by the current state and input character in each iteration.

Despite the characterization as an embarrassingly sequential application, many efforts have been made to parallelize state transitions on FSMs. An intuitive way, referred to as *speculative execution*, involves dividing the input into several parts and performing the computation on different parts speculatively [15, 20, 25, 26]. Since the starting state for a given part is unknown until the execution on the previous part finishes, the algorithm needs to *guess* or *speculate* a starting state for each part. After the execution on the previous part ends, the algorithm checks if the the predicted starting state is identical to the correct state. If not, the computation of that part needs to be re-executed with the correct state. This simple idea has been extended to form more nuanced techniques. For example, looking back characters at the end of preceding part can help improve the prediction [20]. More recently, Zhao *et al.* [25, 26] propose a principled speculation method and eliminate the overheads of offline training in state prediction, making speculative execution of FSMs applicable on the fly.

The problem of speculation is that guessing a good starting state can be expensive, and even impossible in certain cases. Moreover, the overheads of starting speculation and checking correctness can be high in a thread or process-based implementation. Another way to parallelize FSMs is to replace the single-state transition in a straightforward implementation with *enumerative-state* transitions. Enumerative-state transition is a binary operation with the associative property, which allows the entire computation to be parallelized similar to parallel prefix sum [2, 13]. The intuition here is to process an input part with every state in the FSM as the starting state since its correct starting state is unknown. The problem of enumeration is that the redundant computations associated with every state in the FSM can easily outweigh the benefits of parallelization. Mytkowicz *et al.* [17] effectively reduce the redundant work by removing converged states periodically during the computation.

In this paper, we propose an *enumerative speculation* approach that combines and enhances the ideas of speculative and enumerative execution of FSMs. Instead of speculating with one state as in speculative methods, our approach speculates with multiple potential states (but not all states) in the FSM. As the number of guessed states is limited in our approach, the redundant work is limited. Based on the observation of Mytkowicz *et al.* [17] that the number of active (distinctive) states the enumerative execution of a FSM converges in is less than 16 – and even less than 8 almost always. Thus, the motivation behind our approach is that guessing a set of active states is much easier than guessing a single state. Specifically, with a set of states guessed by a simple prediction method, our approach can achieve high speculation success rates while eliminating the need for time-consuming profiling and training procedure(s).

The existing efforts on speculative execution of FSMs have been at thread or process level, exploiting multiple cores or MIMD parallelism for improving performance. Another form of parallelism has become ubiquitous over the last 2 decades – the SIMD parallelism. The most common SIMD or fine-grained data-parallel hardware, Streaming SIMD Extentions (SSE), has been part of the x86 since 1999. There have been significant recent developments in this area, including increasing width of SIMD lanes and enhanced instruction sets that provide more flexibility. Since the introduction of SSE, it has been widely used for graphics [8], image, video, signal processing [4], and regular scientific and engineering applications [5, 22]. More recent works such as [3, 19] have shown that several irregular applications including graph algorithms and particle simulations can also benefit from SIMD parallelization. For enumerative computation of FSMs, Mytkowicz *et al.* [17] exploit SIMD *gather* operation to accelerate multiple state transitions. However, SIMD itself does not bring parallelization to the computation in their work since the concurrent processing of different input parts are still performed by multiple threads. In fact, the total body of work on parallelizing irregular applications on SIMD is quite limited [3].

In this work, we combine SIMD and multi/many-core parallelism in implementing *enumerative speculation* of FSMs. With execution on a single core, the SIMD vector is divided into *groups*, with each group responsible for a different input blocks, and each unit within a group operating with different speculated values. Since the number of states speculated is always less than the length of SIMD vector, which is typically 16 or 8 in today's architectures, our approach limits the redundant work while fully utilizing SIMD capacity in the (co)processors. With execution on multi/many-cores, different input blocks are processed by different threads in parallel. Similar to conventional thread-based speculative execution, a main thread maintains the correct state of the program, while each of several speculative threads processes different input blocks. SIMD parallelism within each thread is used to execute with different speculated starting states, increasing chances that at least one of the states is the correct one.

We evaluate our method with four FSM applications: Huffman decoding, regular expression matching, HTML tokenization, and Div7. The results show that our enumerative speculation has high speculation success rates and achieved 2.2x to 2.5x speedups over the serial code on a single core of Intel Xeon Phi coprocessor. With the same prediction method, our enumerative speculation approach has a much higher success rate than conventional single-state speculation, and achieves an average 1.6x speedup compared with the best single-state speculation for the four applications. We also show good scalability on many/multi-cores – combined with SIMD parallelization, we have up to 95x speedup with 60 cores on an Intel Xeon Phi, and we outperform enumerative execution by an average of 2x.

2. Background

This section provides background on both speculative and enumerative computation methods for FSM parallelization. In addition, we also discuss the key features of the Intel Xeon Phi coprocessor, a commercial release of the Intel Many Integrated Core (MIC) architecture, and the main platform where we conduct our experiments.

2.1 Speculative Computation of FSM

A finite state machine can be represented as a tuple $(Q, \Sigma, q_0, \delta, F)$, where Q is a finite set of states, Σ is a fixed alphabet, $q_0 \in Q$ is the initial state, $F \subset Q$ is the set of final states, and δ is the transition function with type $Q \times \Sigma \rightarrow Q$. We focus on deterministic FSMs – a finite state machine is *deterministic* if given a state and an input character, which cannot be empty, there is only one output state.

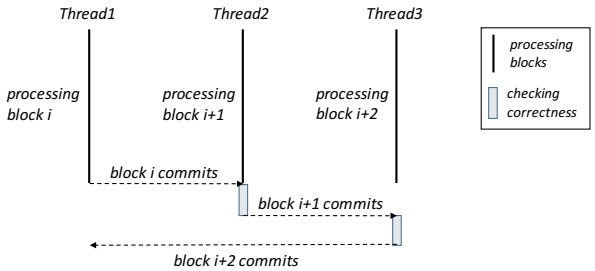


Figure 2: An illustration of speculative computation of FSM

As we stated in the previous section, data dependency is the critical challenge in parallelizing FSM applications. To break the data dependency in state transitions, an intuitive approach is to divide the input data into *blocks*. While one thread is working through a block, another thread can start computing on the next block from what is referred to as a *speculated starting state*. This approach is referred to as *speculative computation* of FSMs [15, 20, 25, 26].

Figure 2 shows an example of speculative computation of FSMs. There are three blocks of input that are processed by three threads. Block i starts from a correct state, while blocks $i+1$ and $i+2$ start from speculated states. When block i finishes the computation, it commits its final state to the block $i+1$. If the speculated starting state of the block $i+1$ is not the same as what block i commits, the computation of the blocks $i+1$ and $i+2$ need to be *rolled back*, and another round of computation for the blocks $i+1$, $i+2$, and $i+3$ will be launched – this time with the block $i+1$ starting from correct state and the other two starting from speculated states. On the other hand, if the guessed starting state of block $i+1$ is identical to what block i commits, then the final state of block $i+1$ is correct and will be committed to the block $i+2$. Now, block $i+2$ repeats the same comparison and so on. If all three blocks are processed successfully, the program will launch another round of computation with the final state of block $i+2$.

The efficiency of the process depends on the success of speculation. Let the size of the *speculation group*, or the number of blocks/threads we attempt concurrent execution on, be denoted by s. Suppose it takes m rounds of execution to finish n iterations, and x_i is the number of iterations computed correctly in the round i. Then, we have, $1 \leq x_i \leq s$, and $\sum_{i=1}^{m} x_i = n$. The best performance is achieved when $x_i = s$ for all iterations, in which case m is the lowest possible, i.e., n/s. The worst case is $m = n$ when all $x_i = 1$. Further, we define the *speculation success rate* α as the number of successful iterations divided by the product of execution rounds and speculation group size, i.e, $\alpha = \frac{\sum_{i=1}^{m} x_i}{s \times m} = \frac{n}{s \times m}$.

2.2 Enumerative Computation of FSMs

As we stated previously, enumerative execution is an alternative to speculative execution. In enumerative execution, instead of computing one state transition, we compute state transitions from every state of the FSM in each iteration. Let $T[c]$ be a state array such that $T[c][i]$ is the state transitioned from state i on the input c. Now, we use $\otimes$ to denote a binary operator such that $i \otimes T[c]$ is $T[c][i]$. Thus, the state transitions in the entire processing of a given string $c_1 c_2 \ldots c_n$ can be represented as a sequence of binary operations:

$$s_{final} = s_{start} \otimes T[c_1] \otimes T[c_2] \ldots \otimes T[c_n]$$

The key idea of enumerative execution is that the binary operator $\otimes$ is associative, and thus, the entire process can be parallelized in the way similar to the parallelization of a prefix sum [2, 13]. A simple implementation involves dividing the input into several blocks and computing $T[c_i] \otimes \ldots \otimes T[c_j]$ for each block concurrently, where c_i and c_j are the start and end character of that block, respectively.

The straightforward implementation described above has a large amount of redundant work, and likely will not deliver speedups in most cases. Mytkowicz *et al.* [17] observed that the states in the enumeration actually converge during the computation. For example, when computing $T[c_i] \otimes \ldots \otimes T[c_j]$, suppose the result of $T[c_i] \ldots \otimes T[c_k] (k < j)$ is S, it is likely that S has a number of *identical states*. Now, only one of these identical states need to be considered in the following computation, i.e, the computation of $T[c_k+1] \otimes \ldots \otimes T[c_j]$. Building further on this idea, Mytkowicz *et al.* propose a *convergence detection procedure*, which can be invoked periodically, to reduce the redundant work. More precisely, an interim result S can be factored as a a pair (L, U) such that $S = L \otimes U$ and U contains only the unique elements of S. A consequence of this is that the convergence detection procedure does not impact the associative property of $\otimes$, and thus the correctness of parallelization is maintained.

2.3 SIMD Instructions

Modern (co)processors have a feature where one can apply an operation on several data elements in a single instruction – referred to as SIMD (Single Instruction Multiple Data) parallelism. The basic SIMD operations include loading/storing data elements from consecutive memory addresses, and adding/multiplying a vector of data elements. In order to apply SIMD parallelism to irregular applications, recent architectures have added more flexible SIMD instructions. Most prominently, *gather/scatter* operations load and store data at non-contiguous memory addresses that are specified by an *index vector*. These instructions can allow loading and storing data for irregular applications where non-contiguous accesses are common. The performance of *gather/scatter* is very close to that of (contiguous) SIMD load/store when the elements to be accessed are close to each other [3], but can be significantly slower otherwise.

These gather and scatter instructions are supported on Xeon Phi processors. Unlike the previous SSE (Streaming SIMD Extension) that was supported on Intel CPUs, Intel Xeon Phi coprocessor has a new 512 bit SIMD instruction set referred to as IMCI or Intel Initial Many Core Instructions. In addition to gather and scatter operations, Intel Xeon Phi also supports a *mask* data type and a set of *mask* operations including *mask-read*, *mask-write*, *mask-add*, and *mask-compare*, which are able to apply an operation on a specified subset of lanes within a SIMD vector.

3. Enumerative Speculation

In the last section, we had described the two sets of existing approaches for parallelizing FSMs, i.e., the speculative and the enumerative methods. It turns out that both the methods have their own drawbacks, which we summarize below.

The speculative methods likely have difficulty in guessing the correct starting state even though sophisticated prediction mechanisms have been developed and used. For example, Div7, which we had shown earlier, is an extreme case in the sense that none of its state pairs converge – in other words, state transitions can be considered *input-insensitive*. More precisely, any state in the FSM can be reached with equal probability on a given input string from different starting states. Thus, profiling the FSM cannot determine which state is more likely to be reached without knowledge of the starting state. The prediction techniques, which are typically based on profiling the FSM and training with inputs (online or offline), can very well fail and result in low speedups in this case, as has been acknowledged by certain studies [25]. Also, the normal implementation for a speculation system is either thread or process based, and there are overheads in checking conflicts and rollback for both thread and process implementations. Since the processing of one input character is simply one read operation, the block length needs to be long enough so that most of the processing cycles are spent on computing rather than on checking the conflicts. This, however, implies larger rollback overheads.

The enumerative methods, on the other hand, have the overhead of redundant computations. Every possible state needs to be computed for the next state, making the amount of computation proportional to the number of states in the FSM. When the FSM is large, the computation can be hundreds (or even thousands!) times higher than a straightforward serial implementation. Mytkowicz *et al.* [17] focus on eliminating this redundancy, observing that the states are actually converging during the enumerative computation in many real applications. Exploiting this property, they devise a redundancy checking and elimination procedure that is invoked periodically during the computation. However, the redundancy eliminating procedure can itself be time-consuming because it relies on a *factor* operation that is not supported in architectures currently.

In this section, we first describe our *enumerative speculation* idea that overcomes the drawbacks of both speculation and enumeration by combining them. Next, we show the benefits of enumerative speculation for increasing speculation success rates – we present a simple method to predict starting states that obtains high speculation success rates with enumerative speculation. Moreover, by evaluating enumerative speculation on different types of FSMs, we observe two important properties for parallelizing FSMs: 1) there is a tradeoff between the redundant work and the parallelism (speculation success rate) for FSMs, and 2) the speculation success rate is insensitive to the block size with multiple speculating states.

3.1 Speculating with Multiple Starting States

In speculative parallelization, an unknown variable is usually assigned only one guessed value for execution. Intuitively, if the program tries multiple guessed values for an unknown variable, it is more likely to get a correct speculation, as compared to only trying one guessed value. Based on this simple observation, we propose the concept of *enumerative speculation*. The idea is to speculate with multiple guessed values and select the correct one from the different speculative executions (or rollback when none of them is correct). More specifically, enumerative speculation uses multiple values as the starting state of an input block and executes with each of these guessed values.

Now, enumerative methods can be viewed as an extreme case of enumerative speculation since every state of the FSMs is tried for each input block. The difference is that there is no speculation in enumerative methods because there must be a correct starting state in all of the states of a FSM. Thus, there is no rollback in enumerative methods. Our method, enumerative speculation, has the advantage that the amount of redundant work is reduced. However, like any speculative method, we may be required to rollback. A key enabler to our method work is a simple method to predict starting states. This method, presented in next subsection, can achieve high speculation success rates in FSM applications with no more than 16 guessed states.

We observe that enumerative speculation is well suited for SIMD execution. With 16-way parallelism, we can easily work with multiple different guessed states. Note that the current state-of-the-art is that SIMD lanes are simply unused for an irregular application like FSM. Thus, achieving a factor of 2 or 3 speedup over serial execution will still be a significant improvement.

3.2 Predicting Starting States

Previous studies on speculative execution of FSMs [20, 26] have shown that a *lookback* technique can improve speculation success rate in applications such as Huffman decoding and XML parsing. The idea of a lookback approach is to examine several characters at the end of the preceding block, perform state transition from an arbitrary state on these characters and choose the resulting final state after these transitions as the guessed starting state for the next block. Not surprisingly, this heuristic performs poorly on a FSM with a large number of states and/or if the states do not converge (as is the case with as Div7 – Figure 1a). Zhao *et. al* [26] attribute the inconsistent performance of lookback approach to two factors. First, the length of suffix to examine impacts the performance – a longer suffix usually produces better guessed state but also causes more overheads. Second, the state used for lookback affects the performance – a good state for lookback can lead to correct guess while a bad one does not. Zhao *et. al* [26] introduce an all-state lookback scheme and propose a *principled speculation* approach based on the observation that the likelihoods for different states to be the actual starting state at a speculation point are usually non-uniform. Though their approach effectively improves the performance of speculation in many applications, it incurs more overheads for online and offline learning.

Now, intuitively, enumerative speculation should avoid these problems, since the probability of a successful speculation is the sum of the likelihoods of each state in the set of guessed states to be the actual starting state. We simplify the all-state lookback approach in [26] to produce a set of guessed states that is very likely to contain the correct starting state. Let v_0 be the vector of all states in an FSM and v_i be the vector of resulting states obtained from v_0 on the input character i, i.e., $v_i[j] = T[i][v_0[j]]$, where T is the transition table. We define the likelihood for $v_i[j]$ to be the correct starting state after input character i, denoted as $L_i[j]$, as its frequency of appearance in v_i. More specifically, suppose the number of states in an FSM is m and $v_i[j]$ appears t times in v_i, then $L_i[j] = t/m$. Similarly, suppose the vector of result states transited from v_0 on input characters a and b is v_{ab}, i.e., $v_{ab}[j] = T[b][T[a][v_0[j]]]$. Now, $L_{ab}[j]$, which is the likelihood for $v_{ab}[j]$ to be the correct starting state after

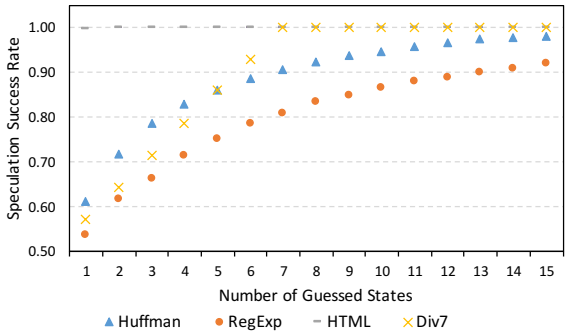


Figure 3: Speculation success rates for four applications: Huffman Decoding, Regular Expression Matching, HTML Tokenization, and Div7 (varying number of guessed states obtained by looking back two input characters)

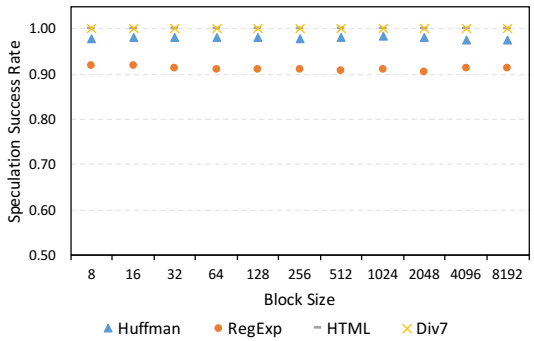


Figure 4: Speculation success rates for four applications: Huffman Decoding, Regular Expression Matching, HTML Tokenization and Div7 (using different block sizes and 15 guessed states)

input character a and b, is p/m, where p is the number of appearance of $v_{ab}[j]$ in v_{ab}.

The single-state speculation approach assumes that the likelihoods for different states to be the correct starting states are non-uniform, so that the most likely one has a high probability of success. In comparison, multiple-state speculation does not require the most likely one to have a high probability. Instead, it assumes that the sum of the few most likely ones have a high probability. Figure 3 shows the speculation success rates of four applications: Huffman decoding (*Huffman*), regular expression matching (*RegExp*), HTML tokenization (*HTML*) and Div7, with different number of guessed states obtained by *lookback-2*, which calculates the likelihoods of states by examining two preceding characters. Here, the number of states is 100+, 200, 38, and 7, respectively, for the four applications. The results show that with only one guessed state, lookback-2 achieves good speculation success rate only for HTML tokenization, and is not effective in the case of the other three applications. Despite the different size of FSMs, the speculation success rates of the other three applications increase as more guessed states are used. All of the four applications achieve speculation success rates of more than 90% when using 15 guessed states.

Figure 4 shows the speculation success rates of four applications running with different block sizes using 15 guessed states produced by lookback-2. We also find that block size has no effect on the speculation success rates, i.e, the set of the 15 states of greatest likelihoods calculated by multiple lookback-2 has a stable and high probability of containing the correct starting state independent of the length of the input characters. These results validate that lookback-2 is effective in achieving good speculation success rates in enumerative speculation.

4. SIMD Implementation of Enumerative Speculation

In this section, we first discuss about how state transitions can be accelerated using SIMD features. Next, we describe a SIMD implementation of enumerative speculation that performs speculation across input blocks. Finally, we discuss an important optimization, which is block size selection.

4.1 SIMD State Transitions

Consider the straightforward FSM implementation shown in Figure 1c. A state is represented by an integer and state transition is implemented by reading a new state from the appropriate position in the transition table (based on the current state and the input character). There are two parts to the computation involved in the process of state transition – the first is the calculation of memory address based on the current state and the input character, and the second is the actual memory access to the transition table based on the calculated address. Assuming we have speculated correctly and are executing different blocks concurrently, the first part can be perfectly parallelized with SIMD and gain speedup proportional to the width of SIMD vector. For example, with a state vector $vstate$, an input vector $vinput$, and a transition table of m states and n input characters as shown in Figure 5, the position of the new state for $vstate[i]$ will be $vinput \times m + vstate$. (assuming characters correspond to the rows and the states correspond to the columns in the transition table).

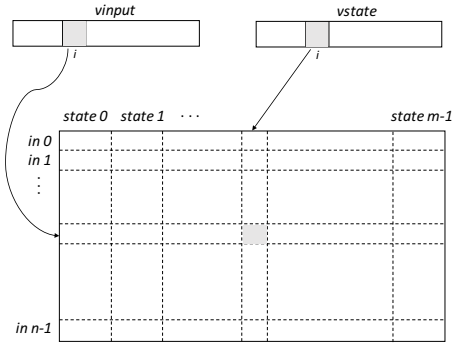


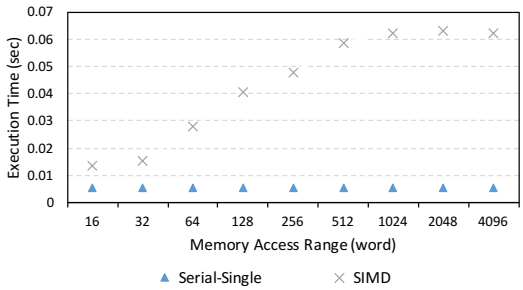
Figure 5: An example of SIMD state transition

The second part, i.e., the actual lookup, can also benefit from SIMD operations. However, the performance now depends on the memory access patterns, as we are loading a set of values using a *gather* operation. To study how this step will perform, we did the following experiment. From the loop we are focusing on, we remove the address calculation part and make the memory accesses to be based only on the current state. In other words, we have a loop that starts from an initial state and uses the current state as the address in an array to access the next state. A version of such a loop,

```
// serial-single version
int x = 0;
for(int i=0;i<N;i++) {
  x = A[x];
}

// SIMD version
__m512i vx = {15,14,…,0};
for(int i=0;i<N;i++) {
  vx = _mm512_i32gather_epi32(vx, A, 4);
}
```

(a)



(b)

Figure 6: Three versions of state transition and their execution time with different memory access ranges

which we refer to as the *serial-single* version, is shown in Figure 6a – here, A is an array of size r comprising random integers ranging from 0 to $r-1$. The SIMD version of the same loop, also shown in Figure 6a, uses SIMD gather operation to perform state transitions for 16 states in a SIMD vector in each iteration.

Figure 6b shows the running time of these two versions, reporting execution over one million iterations on an Intel Xeon Phi – we vary the *memory access range* for each version. We observe that a SIMD gather is as fast as a single load operation when the memory access range is small. Thus, if multiple state transitions in FSMs can be performed *independently* in SIMD and the memory locality is good, FSMs can be accelerated.

4.2 SIMD Speculation

In the existing research, speculative parallelization of FSMs are executed across cores. In such cases, one speculates with a single guessed state for each block and execution starting with the speculated value is processed by a thread or a process. With SIMD state transition described above, and using our idea of enumerative speculation, one can consider a different approach. We can put the determined state for the block i in the first lane of a SIMD vector and several speculated starting states for the block $i+1$ in the rest of lanes. In each step, a single state transition for block i is performed in the first lane with correct state on the current input character, while multiple state transitions on block $i+1$ are performed in other lanes with several speculated states, but using the same input character on each. Because there is no dependence among the lanes in the SIMD vector, all this work can be performed simultaneously. After all of the input characters in the block i are processed, the final state of block i is the correct starting state for the block $i+1$. The program needs to check if the final state of block i is in

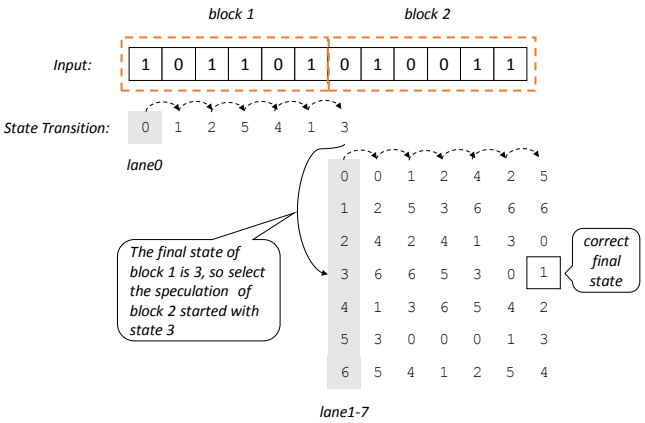


Figure 7: An example of SIMD speculation on FSM for determining if a binary number is divisible by 7

the set of speculated states for the block $i+1$, and if so, the program needs to find the lane with the correct speculation.

Figure 7 shows an example of SIMD speculation on the FSM in Figure 1a (the one that determines if binary number 101101010011 is divisible by 7). Assume that the SIMD vector length is 8 and the input binary string is divided into two blocks. The first lane stores the starting state for the block 1, which is known to be 0. The rest of lanes store the speculated starting states for the block 2, which is the set (0, 1, 2, 3, 4, 5, 6). In this example, for simplicity of demonstration, the guessed starting states happen to be all of the states in the FSM, but in most real cases the guessed starting states are obtained using the multiple lookback approach described in Section 3.2. The program processes block 1 and block 2 simultaneously. The state in the first lane will be 1 after reading the bit "1", while the states in the rest of lanes will turn into the set (0, 2, 4, 6, 1, 3, 5) after reading the bit "0". In each iteration, one bit each in block 1 and 2 are processed. The final state of block 1 is 3. So, the correct final state after processing the entire binary number is the final state of block 2 obtained with starting state of 3, which is 1. Therefore, the binary number is not divisible by 7.

The SIMD speculation described above, in which one SIMD vector performs *speculative* state transition for exactly one block, is called *1-way* SIMD speculation. Our idea of enumerative speculation also allows for alternatives to this approach. Specifically, with wide SIMD vectors, it is even possible to divide the vector lanes into more than one *speculation sets*. For example, the first lane can be used for correct execution of the block i, while half of the remaining lanes can be used for the block $i+1$ and the other half used for the block $i+2$. In this case, three blocks are processed simultaneously, and we call it *2-way* SIMD speculation. Dividing the vector lanes into more than one speculation sets (*k-way* SIMD speculation) provides the opportunity to obtain higher speedup, while also requiring a more complicated logic for checking the correctness of speculation. In the extreme case, a vector of w lanes can perform state transitions for w blocks simultaneously with a single guessed state for each block. This is an extreme case of enumerative speculation, making it identical to simple speculative execution.

4.2.1 Correctness Checking Procedure

We now describe the details of the *correctness checking* procedure for checking the success of speculation. Suppose we are processing block i with a correct state and execution block $i+1$ with a set of speculated states. Let vs be the *starting vector*, i.e., a vector with first element being the actual state for the block i and the remaining elements being the speculated states for the block $i+1$. Similarly, let vf be the *final vector*. Suppose the vector width is w. Now, the correctness checking procedure needs to check if $vf[0]$, which is the final state of block i, is in $vs[1 \ldots w-1]$, which is the set of speculated states for the block $i+1$.

We implement it as follows. Intel MIC architecture supports *mask* data type and SIMD comparison operations. A general approach of correctness checking is to generate a vector with values of $vf[0]$ repeated, and compare it with vs, as shown in Figure 8. The result is a SIMD *mask*, in which each bit indicates whether the corresponding lanes in the two vectors are equal. If all bits (not including the first bit) in the result *mask* are 0, then none of the guessed states is correct. Otherwise, the position of the non-zero bit in the result *mask* is the place of the correct guessed state in vs and the correct final state in vf for the block $i+1$. Though the position can be calculated by $O(w)$ shift operations, an even faster way is to store the positions in an array indexed by the result *mask* and read the value directly from the array.

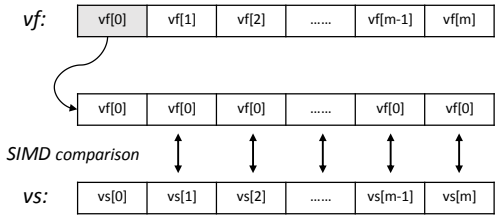


Figure 8: SIMD correctness checking with mask operations

Correctness checking for k-way speculations is similar to that for 1-way speculation. The correctness checking procedure needs to find $vf[0]$, which is the the final state of block i, in the first group of speculation states in vs. If the final state of block i is in the first group, then the corresponding value in vf will be the correct final state of the block $i+1$. With the final state of block $i+1$, the procedure continues to check if the speculation for the block $i+2$ is correct by comparing it with the second group of speculated states. The procedure stops when it finds the first incorrect speculation or if all of the speculations are correct.

4.2.2 Starting A New Speculation

Once the correctness checking procedure determines whether the current round of speculated state has been successful, the program needs to start a new round of speculation. If the speculation for the block $i+1$ is successful, its final state will be available when we are done processing the block i. The next round of speculation will process the block $i+2$ while speculating the block $i+3$ with vs. More specifically, the program will load the final state of the block $i+1$ into the first lane of vs and the guessed states for the block $i+3$ produced by *lookback* into the rest of lanes. If the speculation for the block $i+1$ fails, the program needs to *rollback* and reprocess block $i+1$ with the correct starting state, which is the final state of block i. The program will load the final state of block i into the first lane of vs and the speculated states for the block $i+2$ in the remaining lanes. The next round of speculation will process the block $i+1$ while speculating the block $i+2$ with vs.

For starting new speculation in k-way speculations, suppose the number of speculation sets in a vector is k, the program needs to load the final state of the last block that is correctly speculated, say block g, into the first lane of vs, and load the guessed states for the next k blocks into the rest of lanes. The next round of speculation will process the block $g+1$ while speculating the blocks $g+2$ to $g+k+1$ with vs.

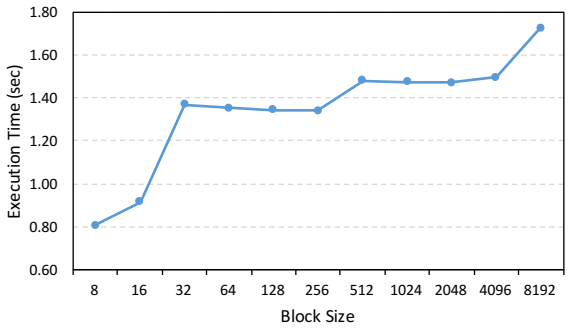


Figure 9: Execution time of reading two consecutive blocks in a text of fixed size simultaneously using SIMD gather with different block sizes

4.3 Selecting Block Size

Though block size does not change speculation success rates, as shown in Section 3.2, it can impact speculation performance in two ways. First, a small block size can lead to better cache performance when reading input text – this is due to hardware prefetching. Second, as there are overheads of predicting starting states and correctness checking for each block, a larger block size implies fewer blocks and thus lower overheads. Figure 9 shows the running time of a program that simulates reading two consecutive blocks simultaneously, varying block size but keeping the total input size the same. We use SIMD gather on Intel Xeon Phi coprocessor. As the block size increases, the time for reading the entire input text increases, and in fact, the execution with a block size of 8 can be twice as fast as the execution of larger block sizes. However, in our evaluation, the programs using multiple lookback-2 prediction have poor performance with a block size of 8, and the best performance is always achieved with a block size between 1024 to 8192. It is because that the overhead of examining 2 characters out of 8 for predicting starting states outweigh the benefit of better cache performance. In cases where the guessed states are simply assigned in order or selected arbitrarily, there is no overhead for predicting starting states, and the logic for checking correctness is simple, a block size of 8 can always achieve the best performance.

5. MIMD Parallelization

Our discussion in the previous section focused on SIMD implementation. Our enumerative speculation method for FSMs can also easily benefit from many/multi-cores, using multiple threads to process different input blocks. We now describe some of the details of the implementation here.

Since the starting state for the first input block is known, the thread for processing the first input block executes the same way as in a single-threaded implementation. Specifically, the first thread divides the first input block into multiple sub-blocks and applies enumerative speculation as described in Section 3. The starting states for the other input blocks are unknown, so the threads for processing these input blocks need to speculate with multiple starting states. The number of states for speculation is usually the length of a SIMD vector so that multiple state transitions can be done by a single SIMD instruction.

Among the different threads, the speculative execution on multiple input blocks is done in parallel, while the correctness checking is sequential. When the correct final state of the block i is available, the program checks if that state is among the set of speculated states the execution of block $i+1$ started with. If so, the final state of the block $i+1$ is obtained and the program proceeds to the block $i+2$. Otherwise, the program needs to reprocess the block $i+1$ with the correct final state of the block i.

A thread can also aggressively process several input blocks simultaneously with k-way speculations. Specifically, the SIMD lanes can be divided into k groups, with each group storing the speculative states for each input block. For example, on a 16-lane SIMD architecture, 2-way speculation will process two input blocks in one SIMD vector with the first 8 lanes being the speculative states for the block i and the other 8 lanes for the block $i+1$.

6. Evaluation Results

6.1 Applications and Baseline Versions

In this section, we evaluate the effectiveness and efficiency of our enumerative speculation for FSM parallelization using four popular FSM-based applications: Huffman decoding (*Huffman*), regular expression matching (*RegExp*), HTML tokenization (*HTML*), and *Div7*. For each application, we implemented a straightforward serial version, a single-threaded 1-way enumerative speculation version, and several k-way enumerative speculation (k>1) versions for both single core and many-core executions.

In addition, to show the advantages of our approach over (pure) speculation, we implemented a single-state speculation version (`PureSpec`) to compare against single-threaded enumerative speculation (we are not aware of any work applying speculation for combining multi-threaded and SIMD parallelization). The pure speculation versions also perform speculation within SIMD vectors and use the lookback-2 prediction – however, only one SIMD lane is used to process the block with a single speculated state. For example, with 1-way SIMD speculation, enumerative speculation uses the first lane for correct state of block i and all of the remaining lanes to execute with 15 different speculated states for the block $i+1$, while pure speculation uses the first lane for the correct state of block i and the second lane storing one guessed state for block $i+1$.

Next, to show the benefits of our approach over (pure) enumeration, we created a version with convergence optimization (referred to as `EnumConv`), following details from the paper by Mytkowicz *et al.* [17]. This version starts execution with all states in the FSMs, and after a certain number of iterations invokes a convergence checking procedure to eliminate redundant states in execution, and subsequently processes with the remaining *active states*. SIMD instructions are used to perform multiple state transitions to hide the overhead of enumeration. As different applications achieve best performance with convergence checking after different numbers of iterations, this parameter was manually tuned to obtain the best performance for comparing against our approach. For all applications, our comparison against pure speculation is with a single-core execution, whereas comparison against enumeration is on single and multi/many-cores.

Our experiments are conducted on an Intel Xeon Phi SE10P coprocessor which has 61 cores (1.1 GHz) with each core incorporated with 512-bit SIMD units. We use Intel ICC compiler 13.1.0 to compile all the codes with -O3 optimization enabled. For the applications where we experimented with k-way speculation, the vector lanes are divided evenly into k groups. For example, with 2-way speculation, the vector lanes are divided as 1-8-7 on single core, which means the first lane processes with the correct state, the following eight lanes process the next block with eight different speculated values, and the remaining seven lanes process yet another block with seven different speculated states for the second speculation. For 2-way speculation with multi/many-cores, the vector lanes are simply divided as 8-8. For higher values of k, vector lanes are divided as 1-5-5-5 in 3-way SIMD speculation and as 1-3-2-2-2-2-2-2 in 7-way speculation on single core, and divided as 6-5-5 in 3-way and 3-3-2-2-2-2-2 in 7-way with multi/many-core execution. The extreme case is single-core 15-way SIMD speculation where each lane is used for conducting state transition on a different input block.

6.2 Huffman Decoding

Huffman coding is widely used in text compression. It utilizes non-uniform frequencies of characters found in texts and encodes the characters to variable-length bit strings. The mapping from characters to the variable-length bit strings is obtained by the *Huffman encoding procedure*, which is a greedy algorithm that builds a Huffman tree. Huffman decoding, which is the reverse process of Huffman encoding, interprets the bit strings back to the characters based on the Huffman tree – this process is very similar to the process of executing state transitions in an FSM. Thus, we have experimented with the decoding part of this method. The input texts we use are from the most downloaded books from the Project Gutenberg[1]. Table 1 lists the names of the downloaded text files and the number of states in their corresponding FSMs built by Huffman encoding. The number in the filename is the index of that book on the website.

File Name	Number of States in FSM
76.txt.utf-8	179
50247.txt.utf-8	203
98.txt.utf-8	177
74.txt.utf-8	179

Table 1: Input texts used in evaluating Huffman decoding with the number of states in their corresponding FSM built by Huffman encoding

Figure 10 shows the execution time of a straightforward serial implementation (`Serial`) and three versions that use SIMD lanes. These three versions are 1-way enumerative

[1] `http://www.gutenberg.org/`

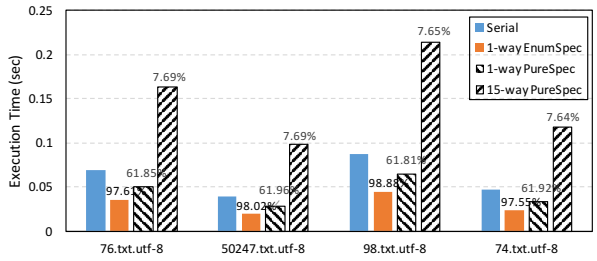


Figure 10: Execution time of different versions of Huffman decoding on a single core (raw numbers are the speculation success rate)

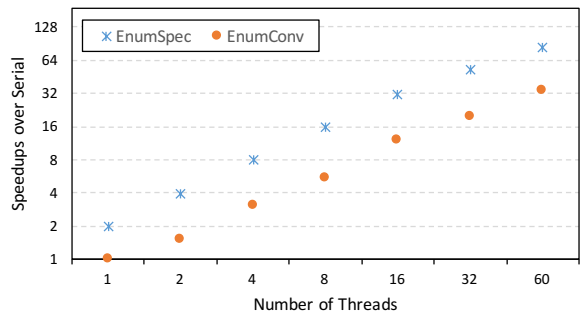
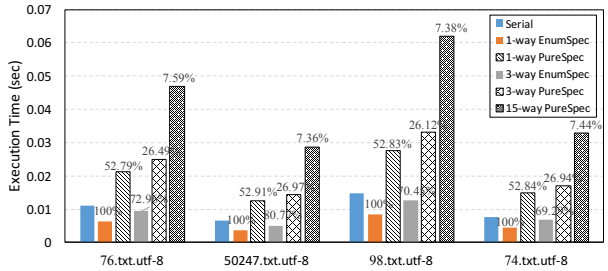


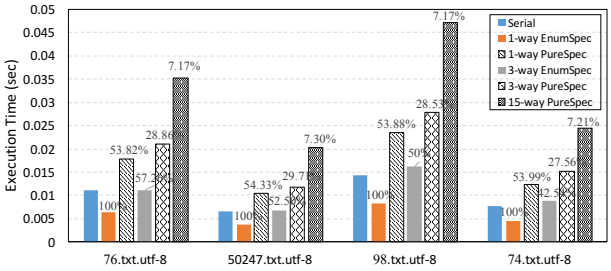
Figure 11: Speedups over serial code – Huffman decoding with 2-way speculation and pure enumeration

speculation version (`EnumSpec`), a 1-way pure speculation version (`PureSpec`) (which uses only 2 SIMD lanes) and a 15-way pure speculation. We did not use k-way speculation as it had lower performance. Also, comparison against enumeration will be reported later. The speculation success rates of each execution, calculated as in Section 2.1, are also labeled on the top of each column. The 1-way enumerative speculation reaches 2x speedup over serial version on all of the input texts. The 1-way pure speculation has a lower success rate, and thus, runs slower than 1-way enumerative speculation. The 15-way runs even much slower than the serial code because the prediction of speculative states incurs overheads while the speculation success rate with lookback-2 is quite low.

For evaluating many-core performance, we use a 100 MB input text so that the input block per core is large enough to justify the cost of invoking more threads. We implemented a multi-threaded version (`EnumSpec`) with 2-way speculation (each thread processes two input blocks simultaneously) and lookback-4 prediction (looking back four preceding characters) – this combination had the best performance among all options. Figure 11 shows the speedups of multi-threaded 2-way enumerative speculation (and 1-way enumerative speculation with a single core). Both x and y-axis in the figure are formatted as base-2 logarithmic scale. The multi-threaded 2-way enumerative speculation reaches 99.99% speculation success rate with lookback-4 prediction. It has linear speedups with increasing number of cores and can achieve $2n$ times speedup with n cores over the serial code. We also see that `EnumConv` also has a near-linear speedup with increasing number of cores, but is slower than our approach by a factor of 2.



(a) Results for regular expression$(.*l.*i.*k.*e)|(.*a.*p.*p.*l.*e)$



(b) Results for regular expression $(.+,.+\backslash.)^4|(.+,)^4|(.+\backslash.)^4$

Figure 12: Execution times of different versions of Regular Expression Matching

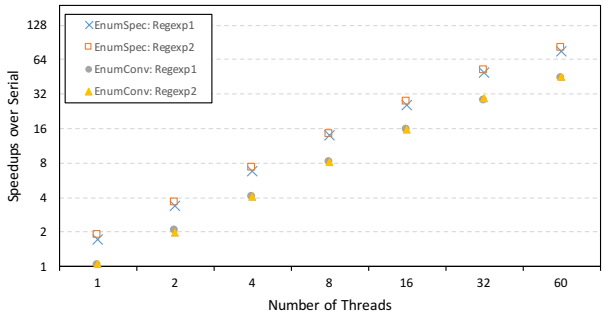


Figure 13: Speedups over serial code for regular expression matching – 4-way speculation and pure enumeration

6.3 Regular Expression Matching

The application we experimented with reads in a text file and searches for all the strings that matches a given pattern. Once a match is found, the program outputs the position of the match in the text file. For comparisons on a single core, we implemented enumerative speculation and pure speculation versions – both 1-way and 3-way – as well as a 15-way pure speculation. The input texts we use here are the same as in Huffman decoding (listed earlier in Table 1). We test the performance with two regular expressions: "$(.*l.*i.*k.*e)|(.*a.*p.*p.*l.*e)$" and "$(.+,.+\backslash.)^4|(.+,)^4|(.+\backslash.)^4$", which are used also by Zhao *et al.* [26]. The "." in the patterns represents any character, "\." represents the period, and the superscripts indicates repetitions.

The results with two regular expressions are shown in Figures 12a and 12b, respectively. The 1-way enumerative speculation has a 100% success rate and reaches 1.73x speedup over the serial version with both regular expressions. In comparison, 1-way pure speculation has a much

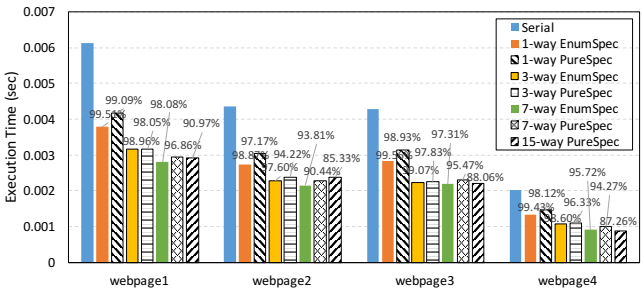


Figure 14: Execution times of different versions of HTML tokenization on a single core

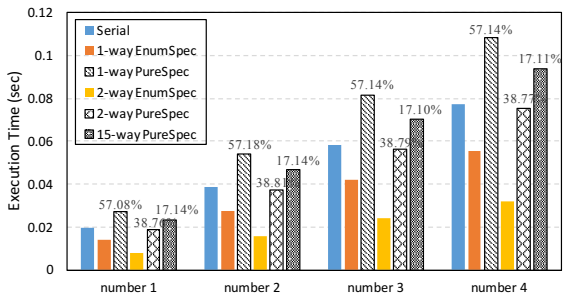


Figure 15: Execution time of different versions of Div7 on a single core

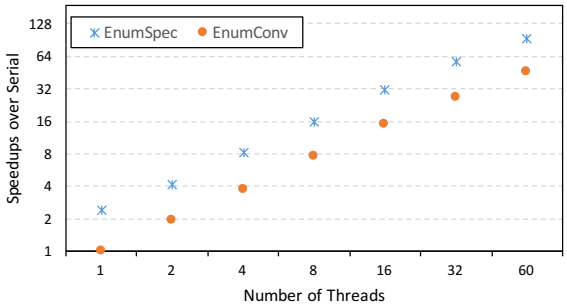


Figure 16: Speedups over serial code – Div7 with 2-way speculation and pure enumeration

lower success rate (53%) and runs even slower than the serial code. The speculation success rate decreases with 3-way speculation for both enumerative and pure speculation. With 3-way enumerative speculation, our approach still outperforms the serial code on the first regular expression, and is slightly slower than the serial code on the second regular expression. The 15-way pure speculation has the worst performance among all of the six versions due to a low speculation success rate.

For evaluating performance on many-cores, we implemented a multi-threaded version (`EnumSpec`) with 2-way speculation (each thread processes two input blocks simultaneously) and lookback-2 prediction. We match the two regular expressions above with a 1GB input text. Figure 13 shows the speedups (for 1-core execution, we use 1-way enumerative speculation). With lookback-2, the multi-threaded 2-way enumerative speculation reaches 100% speculation success rate on both regular expressions. The results show that our approach scales well on many-cores, and we achieve at least $1.6n$ times speedup with n cores over the serial code. The speedups of pure enumeration is near linear on many-cores, but is also a factor slower than our approach due to weak performance on a single core.

6.4 HTML Tokenization

HTML tokenization is widely used in web browsers and web crawlers to extract HTML tokens. Our implementation uses a FSM with 38 states, and the texts used in our evaluation are web pages from New York Times website. For single core performance comparisons, We also implement single-threaded 1-way, 3-way, 7-way, and 15-way SIMD speculation versions.

Figure 14 shows the running time of the different versions on a single core, using different web pages. The 1-way enumerative speculation has 1.5x to 1.6x speedup over the serial version. The speculation success rates in 3-way and 7-way progressively decrease as fewer number of lanes are used for each speculation, but the program runs faster as more input blocks are processed simultaneously. The pure speculation versions have lower speedups due to slightly lower speculation success rates. The success rates of pure speculation turn out to be close to those of enumerative speculation because the states tend to converge to the state 0, and thus, prediction is easy. The performance of 7-way enumerative speculation is the best for most inputs, achieving 1.96x to 2.21x speedups over serial code. The 15-way pure speculation, which uses only one guessed state for each input block, still keeps high speculation success rates (about 90%) and achieves a comparable speedup with enumerative speculation. Note that for this application, we do not evaluate the performance of HTML tokenization on many-cores because webpages are usually small (less than 1 MB).

6.5 Div7

The FSM that Div7 uses was shown earlier in Figure 1a. Though there are only seven states in the FSM, it is hard to predict the starting state for a given block. However, with our enumerative speculation, it is easy to put all the states to a SIMD vector (of length 8 or 16), and the speculation success rate of 100% is easily obtained. We implement a serial version similar to Figure 1c as the baseline, and single-threaded 1-way and 2-way enumerative and pure speculation for comparison. The binary numbers given to the programs are randomly generated, and are of varying length.

Figure 15 shows the running time of the six versions on a single core of Intel Xeon Phi. The 1-way enumerative speculation has 1.85x speedup over serial version, whereas 2-way enumerative speculation achieves speedups up to 2.45x. Both 1-way and 2-way enumerative speculation have guaranteed success rate of 100%. The corresponding pure speculation versions have much lower speculation success rates and poor performance.

To evaluate many-core performance, we implemented a multi-threaded 2-way speculation (each thread process two input blocks simultaneously) version (`EnumSpec`) and a pure enumeration version with convergence optimization (`EnumConv`). Since the FSM has only 7 states which can fit in the 16 lanes of SIMD vector, there is actually no speculation in "2-way speculation". Also, since the FSM does not converge, there is also no convergence optimization in the

enumeration. The only difference between our 2-way speculation and the pure enumeration is that our method takes full advantage of the 16 lanes in a SIMD vector for processing two input blocks simultaneously. Figure 16 shows the speedups of Div7 brought by our speculation approach and pure enumeration approach over serial code. It shows that our approach scales well on many-cores and achieves $1.6n$ to $2n$ times speedup with n cores, while the pure enumeration version is slower than our approach by a factor of 2.

7. Related Work

There has been a lot of work on accelerating irregular applications on various parallel platforms, including multi-core machines, GPUs, as well as vector instruction sets. Because of the very large volume of work in this area, this section summarizes closely related efforts on accelerating FSMs (which are one type of irregular applications), and the efforts on mapping irregular applications on the modern SIMD hardware.

Enumerative Methods for FSM As described in Section 2.2, the basic idea here is to apply parallel prefix sum algorithm [2, 6, 13]. However, the challenge is the large amount of redundant work. Several optimizations [7, 17, 18] are proposed to reduce the amount of redundant work. Pan *et al.* [18] construct a powerset of a FSM, which implicitly eliminates the redundant states, to emulate the enumerative computation. As the entire powerset needs to be constructed statically, and the number of states in the powerset can be extremely large, their approach is limited. Holub *et al.* [7] present an optimized implementation based on the assumption that FSM is *k-local*, i.e., any two states in the FSM converge to the same state on any input of length k. However, more recently, Mytkowicz *et al.* [17] have shown that most FSMs seen in practice are not k-local. Instead, their conclusion is that that states in most FSMs converge to no more than 16 distinct states after a certain length of input, and their work includes a periodical convergence checking technique to eliminate the redundancy. They also exploit both SIMD and MIMD units on processor to accelerate the computation, though use of SIMD is limited to gathering inputs only. We have compared our work against an implementation of the technique described in their paper, and shown that we outperform them because of better utilization of SIMD lanes.

Speculative Methods for FSM Speculation is a common strategy to break dependence in computation. Klein *et al.* [12] present a speculative parallelization of Huffman decoding. Luchaup *et al.* [15] propose a speculative parallelization of regular expression matching. Prabhu *et al.* [20] show that a lookback approach can be helpful to predict the starting state in FSMs. Zhao *et al.* [26], however, argue that a simple fixed length lookback approach is not always sufficient. They propose a *principled speculation method* that pre-analyzes FSMs to get the application-specific information and use this information in a probabilistic model. They further propose an *on-the-fly* principled speculation technique [25] to remove the overheads of offline training. Both the efforts still require that the states in an FSM are likely to converge. Also, all of these efforts use multi-threading or multi-processing, and not SIMD. We have implemented speculation on SIMD hardware and demonstrated that our method, enumerative speculation, performs better. More recently, Qiu *et al.* [21] provide a rigorous analysis among three types of parallelism that can be exposed at the fine-grain level for FSMs. Though they discuss using SIMD to conduct multiple speculations, the tradeoff between redundant work and parallelism is not explored in their work.

SIMD Acceleration for Irregular Applications Pennycook *et al.* implemented *Moldyn* on the Xeon Phi, using both MIMD and SIMD parallelism [19]. This is a very initial investigation, using a relatively straight-forward implementation specific to a single application. Saule and Catalyurek [23] provided a preliminary evaluation on graph applications on the Xeon Phi architecture, but without particular optimizations specific to the applications and the hardware. Liu *et al.* [14] used ELLPACK sparse block format, to optimize SpMV kernel on the Intel MIC. Similarly, Tang *et al.* [24] utilized a hybrid storage format with jagged partitioning to optimize SpMV, also for the Intel MIC. Kim *et al.* [11] compile irregular application to use SIMD features of the Cell processor, with vector units that do not support modern SIMD operations such as *gather/scatter*. More recently, Chen *et al.* [3, 9] proposed a tiling-and-grouping approach that improves the data locality and applies SIMD processing in a series of applications including graph algorithms, irregular reduction, molecular interaction and matrix multiplication.

There have been several studies mapping irregular applications to SIMT processing on GPUs. For example, Merrill *et al.* [16] parallelized breadth-first search on the GPUs by focusing on fine-grained task management, and CuSha [10] optimizes graph processing on GPUs with intensive usage of shared memory and by re-organizing the graph data in shards.

8. Conclusions

This paper has presented a novel enumerative speculation method for parallelizing FSM applications. Instead of speculating on a single state, this approach uses multiple states as guessed starting states to achieve higher speculation success rates. We also describe a lookback approach for generating the set of guessed states and show that the approach is efficient and effective. We focus on parallelizing using SIMD instruction sets, as multiple state transitions can be performed simultaneously on different lanes of SIMD vectors, and in combining SIMD processing with multi/many-cores. Our experiments validate that the enumerative speculation method has higher speculation success rates than pure speculation using the same prediction method. The comparison of different versions of SIMD speculation on a single core shows that our approach achieves 2.2x to 2.5x speedups over the serial code and an average 1.6x speedup over the best pure speculation code. Comparison between enumerative speculation and pure enumeration over many/multi-cores shows good scalability of our approach and an average 2x speedup over pure enumeration.

Acknowledgements

This work was supported by NSF award CCF-1526386.

References

[1] K. Asanovic, R. Bodik, J. Demmel, T. Keaveny, K. Keutzer, J. Kubiatowicz, N. Morgan, D. Patterson, K. Sen, J. Wawrzynek, D. Wessel, and K. Yelick. A view of the

parallel computing landscape. *Commun. ACM*, 52(10):56–67, Oct. 2009. ISSN 0001-0782. doi: 10.1145/1562764.1562783.

[2] G. E. Blelloch. Prefix Sums and Their Applications. Technical Report CMU-CS-90-190, School of Computer Science, Carnegie Mellon University, Nov. 1990.

[3] L. Chen, P. Jiang, and G. Agrawal. Exploiting Recent SIMD Architectural Advances for Irregular Applications. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, CGO 2016, 2016.

[4] F. Franchetti and M. Puschel. A SIMD Vectorizing Compiler for Digital Signal Processing Algorithms. In *Proceedings of the 2002 IEEE International Parallel and Distributed Processing Symposium (IPDPS 2002)*, pages 20 –26. IEEE, 2002.

[5] C. Garca, R. Lario, M. Prieto, L. Piuel, and F. Tirado. Vectorization of Multigrid Codes Using SIMD ISA Extensions. *Proceedings of the 2003 IEEE Inernational Parallel and Distributed Processing Symposium (IPDPS 2003)*, 0:58a, 2003.

[6] W. D. Hillis and G. L. Steele, Jr. Data Parallel Algorithms. *Commun. ACM*, 29(12), Dec. 1986.

[7] J. Holub and S. Štekr. On Parallel Implementations of Deterministic Finite Automata. In *Proceedings of the 14th International Conference on Implementation and Application of Automata*, CIAA '09, 2009.

[8] N. Ide, M. Hirano, Y. Endo, S. Yoshioka, H. Murakami, A. Kunimatsu, T. Sato, T. Kamei, T. Okada, and M. Suzuoki. 2.44-GFLOPS 300-MHz Floating-Point Vector-Processing Unit for High-Performance 3D Graphics Computing. *IEEE Journal of Solid-State Circuits*, 35(7):1025 –1033, Jul 2000.

[9] P. Jiang, L. Chen, and G. Agrawal. Reusing data reorganization for efficient simd parallelization of adaptive irregular applications. In *Proceedings of the 2016 International Conference on Supercomputing*, ICS '16, 2016.

[10] F. Khorasani, K. Vora, R. Gupta, and L. N. Bhuyan. CuSha: Vertex-centric Graph Processing on GPUs. In *Proceedings of the 23rd International Symposium on High-performance Parallel and Distributed Computing*, HPDC '14, 2014.

[11] S. Kim and H. Han. Efficient simd code generation for irregular kernels. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '12, pages 55–64, New York, NY, USA, 2012. ACM. ISBN 978-1-4503-1160-1. doi: 10.1145/2145816.2145824. URL `http://doi.acm.org/10.1145/2145816.2145824`.

[12] S. T. Klein and Y. Wiseman. Parallel Huffman Decoding with Applications to JPEG Files. *THE COMPUTER JOURNAL*, 46:487–497, 2003.

[13] R. E. Ladner and M. J. Fischer. Parallel prefix computation. *J. ACM*, 27(4), Oct. 1980.

[14] X. Liu, M. Smelyanskiy, E. Chow, and P. Dubey. Efficient Sparse Matrix-vector Multiplication on x86-based Many-core Processors. ICS '13, 2013.

[15] D. Luchaup, R. Smith, C. Estan, and S. Jha. Speculative Parallel Pattern Matching. *IEEE Transactions on Information Forensics and Security*, 6(2):438–451, June 2011.

[16] D. Merrill, M. Garland, and A. Grimshaw. Scalable gpu graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, PPoPP '12, pages 117–128, New York, NY, USA, 2012. ACM. ISBN 978-1-4503-1160-1. doi: 10.1145/2145816.2145832. URL `http://doi.acm.org/10.1145/2145816.2145832`.

[17] T. Mytkowicz, M. Musuvathi, and W. Schulte. Data-parallel finite-state machines. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14, pages 529–542, New York, NY, USA, 2014. ACM. ISBN 978-1-4503-2305-5.

[18] Y. Pan, Y. Zhang, and K. Chiu. Simultaneous Transducers for Data-Parallel XML Parsing. In *Parallel and Distributed Processing, 2008. IPDPS 2008. IEEE International Symposium on*, pages 1–12, 2008.

[19] S. J. Pennycook, C. J. Hughes, M. Smelyanskiy, and S. A. Jarvis. Exploring simd for molecular dynamics, using intel xeon processors and intel xeon phi coprocessors. In *Proceedings of the 2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, IPDPS '13, 2013.

[20] P. Prabhu, G. Ramalingam, and K. Vaswani. Safe Programmable Speculative Parallelism. In *Proceedings of Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, Inc., June 2010.

[21] J. Qiu, Z. Zhao, and B. Ren. Microspec: Speculation-centric fine-grained parallelization for fsm computations. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*, PACT '16, pages 221–233, New York, NY, USA, 2016. ACM. ISBN 978-1-4503-4121-9. doi: 10.1145/2967938.2967965. URL `http://doi.acm.org/10.1145/2967938.2967965`.

[22] T. Rognes and E. Seeberg. Six-Fold Speed-up of Smithwaterman Sequence Database Searches Using Parallel Processing on Common Microprocessors. 16(8):699–706, 2000.

[23] E. Saule and U. V. Catalyurek. An early evaluation of the scalability of graph algorithms on the intel mic architecture. In *Proceedings of the 2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*, IPDPSW '12, pages 1629–1639, Washington, DC, USA, 2012. IEEE Computer Society. ISBN 978-0-7695-4676-6. doi: 10.1109/IPDPSW.2012.204. URL `http://dx.doi.org/10.1109/IPDPSW.2012.204`.

[24] W. T. Tang, R. Zhao, M. Lu, Y. Liang, H. P. Huynh, X. Li, and R. S. M. Goh. Optimizing and Auto-tuning Scale-free Sparse Matrix-vector Multiplication on Intel Xeon Phi. In *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization*, CGO '15, 2015.

[25] Z. Zhao and X. Shen. On-the-Fly Principled Speculation for FSM Parallelization. *SIGARCH Comput. Archit. News*, 43(1), Mar. 2015.

[26] Z. Zhao, B. Wu, and X. Shen. Challenging the "Embarrassingly Sequential": Parallelizing Finite State Machine-based Computations Through Principled Speculation. *SIGPLAN Not.*, 2014.

A. Artifact Description

A.1 Abstract

The artifact includes all of the programs that are needed to reproduce the evaluation results in the paper. The programs should be executed on an Intel Xeon Phi SE10P coprocessor with 61 cores. An Intel ICC compiler is required for compiling. Python is also require for executing the scripts that generate some of the inputs. The artifact will print out the execution time of different versions of the evaluated applications on different datasets, which can be expected to be close to the results reports. The artifact will also print out the speculation success rates of different versions of applications, and the rates should be exactly the same as reported in the paper.

A.2 Description

A.2.1 Check-list (artifact meta information)

- **Program:** (1) Huffman decoding, (2) Regular expression matching, (3) HTML tokenization, (4) Div7, each stored in a separate directory with different versions
- **Compilation:** Intel ICC compiler 13.1.0
- **Datasets:** four text files, four webpages, and two python scripts that generate some inputs, all included in the artifact
- **Hardware:** an Intel Xeon Phi SE10P coprocessor with 61 cores
- **Output:** program results including the execution time are printed out to the console
- **Experiment workflow:** a bash script is provided to automatically start the compilation and execution, or simply run every version of every application individually
- **Publicly available?:** Yes

A.2.2 How delivered

The artifact is publicly available at `https://github.com/jiangohiostate/ppopp17_artifact.git`. The initial package is about 4MB. After all datasets are generated, it consumes about 1.2GB disk space.

A.2.3 Hardware dependencies

An Intel Xeon Phi SE10P coprocessor with 61 cores is required.

A.2.4 Software dependencies

Intel ICC compiler 13.1.0 and Python2.x are required.

A.2.5 Datasets

All datasets are included in the package, or can be generated by the scripts in the package.

A.3 Installation

In each of the four directories, there is a 'run.sh' script, which builds the programs, generates the inputs, and run the tests automatically. You can also compile the programs with the Makefile that provided in each directory.

A.4 Experiment workflow

There are three steps to conduct the experiments:

1) Compile the programs. This can be done by make in each of the four directories in the package.

2) Generate the input data. The small input files are already included in the package, while the large ones are generated by python scripts, which reside in the 'datasets' subdirectory of each application's directory. The data will be generated by executing the scripts as "python gen_data.py".

3) Execute the programs. Each of the application has a baseline named 'serial', several single-core versions named 'enumspec*/purespec*' and two multi-core versions named 'enumspec_mimd/enumconv_mimd' (except for HTML tokenization). The programs are executed with command line arguments that specify the input files and number of cores to be used. Detailed usage can be found in the 'run.sh' script in the directory.

Alternative to the above three steps, you can simply execute the 'run.sh' script in each directory. It will automatically compile the programs, generate the inputs, and execute the programs.

A.5 Evaluation and expected result

The execution time of the programs will be printed out in the console, and they should be very close to those reported in the paper. The programs will also print out the final state of the FSM, which should be the same for different versions of an application given the same input. It can verify the correctness of the different versions.

The speculation success rates are also printed out, and they should be the same as reported in the paper.

A.6 Notes

Intel Xeon Phi SE10P is a coprocessor. The 'run.sh' scripts in the package assume you have logged in the coprocessor, so that the binaries can be directly executed in the bash.

Submission and reviewing methodology: `http://cTuning.org/ae/submission-20161020.html`